Taller práctico: OAuth 2.0 y OpenID Connect con Spring Security

Guía paso a paso para estudiantes | Nivel básico–intermedio | Duración estimada: 2 a 3 horas

Propósito: implementar una aplicación web que permita iniciar sesión con Google, consultar información de identidad del usuario y proteger una página privada. No se necesita base de datos ni Docker.

1. Resultados de aprendizaje

• Distinguir OAuth 2.0 (autorización delegada) de OpenID Connect (autenticación).

• Configurar un cliente OAuth 2.0/OIDC en Google Cloud y Spring Security.

• Proteger rutas, mostrar datos del usuario autenticado y cerrar la sesión local.

• Verificar el comportamiento de la aplicación y documentar evidencias.

2. Requisitos previos

Java 21 (recomendado) o Java 25 compatible con la versión elegida de Spring Boot; IntelliJ IDEA; acceso a Internet; Maven integrado; cuenta Google; navegador web. Se utiliza Spring Boot 3.5.x, Spring Security 6 y Thymeleaf.

3. Arquitectura y conceptos

Elemento	Función
Navegador	Solicita acceso a /perfil
Spring Security	Inicia el flujo Authorization Code, valida identidad y crea sesión
Google	Proveedor de identidad OIDC
ID Token	Token firmado que contiene claims de identidad
Aplicación	Muestra nombre, correo e identificador del usuario

Secuencia: usuario → aplicación → Google → URI de redirección → sesión autenticada → perfil. OAuth 2.0 define autorización delegada; OIDC añade autenticación e ID Token.

4. Paso 1: crear el proyecto

En IntelliJ IDEA: New Project → Spring Boot (o https://start.spring.io/). Configurar Maven, Java, Group com.ejemplo, Artifact oauth2-demo, Spring Boot 3.5.x y Java 21. Añadir Spring Web, Spring Security, OAuth2 Client y Thymeleaf. Crear y abrir el proyecto. Esperar a que Maven descargue dependencias.

Estructura esperada

```
oauth2-demo/
  pom.xml
  src/main/java/com/ejemplo/oauth2demo/
    Oauth2DemoApplication.java
    config/SecurityConfig.java
    controller/HomeController.java
  src/main/resources/
    application.properties
    templates/index.html
    templates/perfil.html
```

Nota: si el nombre de la clase principal generado por IntelliJ difiere, conservar el generado. Los paquetes del código deben coincidir con el paquete raíz real.

5. Paso 2: configurar Google Cloud

• Abrir https://console.cloud.google.com/ e iniciar sesión.

• Crear un proyecto, por ejemplo «Spring OAuth2 Taller».

• En Google Auth Platform configurar Branding (nombre de la aplicación y correo de soporte), Audience y Data Access si la consola lo solicita.

• Para una aplicación externa en pruebas, agregar como usuarios de prueba las cuentas que utilizarán el taller.

• En Clients, crear un cliente OAuth de tipo Web application.

• En Authorized redirect URIs registrar exactamente:
http://localhost:8080/login/oauth2/code/google

• Guardar y copiar Client ID y Client Secret en un lugar privado.

Importante: el cliente debe ser de tipo aplicación web, no Android ni escritorio. Los cambios de la consola pueden variar con el tiempo. La URI de redirección debe coincidir exactamente, incluido puerto y ruta.

6. Paso 3: configurar propiedades y secretos

Crear o editar src/main/resources/application.properties:

```
spring.application.name=oauth2-demo
server.port=8080
spring.security.oauth2.client.registration.google.client-id=${GOOGLE_CLIENT_ID}
spring.security.oauth2.client.registration.google.client-secret=${GOOGLE_CLIENT_SECRET}
spring.security.oauth2.client.registration.google.scope=openid,profile,email
```

En IntelliJ: Run → Edit Configurations → seleccionar la configuración de ejecución → Environment variables. Crear las variables GOOGLE_CLIENT_ID y GOOGLE_CLIENT_SECRET con sus valores

reales. En Windows se pueden introducir como pares separados por punto y coma desde el editor de variables. Reiniciar la aplicación después de modificarlas.

No incluir Client Secret ni tokens en capturas, repositorios o entregas. No subir archivos .env que contengan credenciales.

7. Paso 4: configurar Spring Security

Crear config/SecurityConfig.java:

```
package com.ejemplo.oauth2demo.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {
    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
            throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/error").permitAll()
                .requestMatchers("/perfil").authenticated()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth -> oauth
                .defaultSuccessUrl("/perfil", true)
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/")
                .permitAll()
            );
        return http.build();
    }
}
```

Interpretación: permitAll permite la página de inicio; authenticated protege /perfil; oauth2Login habilita el inicio de sesión OIDC; logout cierra la sesión local. Se conserva la protección CSRF predeterminada.

8. Paso 5: crear el controlador MVC

Crear controller/HomeController.java:

```
package com.ejemplo.oauth2demo.controller;

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.oidc.user.OidcUser;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class HomeController {
    @GetMapping("/")
    public String inicio() {
        return "index";
    }

    @GetMapping("/perfil")
    public String perfil(@AuthenticationPrincipal OidcUser usuario,
                         Model model) {
        model.addAttribute("nombre", usuario.getFullName());
        model.addAttribute("correo", usuario.getEmail());
        model.addAttribute("foto", usuario.getPicture());
        model.addAttribute("identificador", usuario.getSubject());
        return "perfil";
    }
}
```

OidcUser representa al usuario autenticado por OIDC. El subject (sub) es el identificador del usuario emitido por el proveedor; no es una contraseña.

9. Paso 6: construir la página pública

Crear src/main/resources/templates/index.html:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Inicio | OAuth2</title>
</head>
<body>
    <h1>Taller Spring Security</h1>
    <p>Autenticación con OAuth 2.0 y OpenID Connect</p>
    <a href="/oauth2/authorization/google">Iniciar sesión con Google</a>
</body>
</html>
```

El enlace /oauth2/authorization/google inicia el flujo con el proveedor registrado como google.

10. Paso 7: construir el perfil protegido

Crear src/main/resources/templates/perfil.html:

```
<!DOCTYPE html>
<html lang="es" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Mi perfil</title>
</head>
<body>
```

```
    <h1>Perfil autenticado</h1>
    <h2 th:text="${nombre}">Nombre</h2>
    <p>Correo: <span th:text="${correo}"></span></p>
    <p>ID OIDC: <span th:text="${identificador}"></span></p>
    <img th:if="${foto != null}" th:src="${foto}"
         alt="Foto de perfil" width="100">
    <form th:action="@{/logout}" method="post">
        <button type="submit">Cerrar sesión</button>
    </form>
</body>
</html>
```

El formulario POST de logout utiliza el mecanismo CSRF integrado con Thymeleaf y Spring Security. No cambiarlo por un enlace GET.

11. Paso 8: ejecutar y verificar

• Ejecutar Oauth2DemoApplication desde IntelliJ (botón Run).

• Comprobar en la consola que el servidor inicia en el puerto 8080.

• Abrir http://localhost:8080 y seleccionar «Iniciar sesión con Google».

• Elegir una cuenta habilitada como usuario de prueba, si corresponde.

• Comprobar que se muestra /perfil con nombre, correo e identificador.

• Seleccionar «Cerrar sesión» y comprobar que regresa a la página pública.

Si el puerto 8080 está ocupado, cerrar el otro proceso o cambiar el puerto; en ese caso, actualizar también la URI autorizada en Google Cloud.

12. Pruebas funcionales obligatorias

ID	Acción	Resultado esperado
P01	Abrir /	Página pública visible
P02	Abrir /perfil sin sesión	Redirección al inicio de autenticación
P03	Iniciar sesión con Google	Regreso a /perfil autenticado
P04	Ver perfil	Nombre, correo y subject visibles
P05	Cerrar sesión	Regreso a /
P06	Abrir /perfil tras logout	Se requiere autenticación de nuevo

Nota: la sesión de Google puede continuar activa después del logout local, por lo que una nueva autenticación podría completarse sin pedir contraseña.

13. Problemas frecuentes y solución

Síntoma	Posible causa y corrección

redirect_uri_mismatch	Revisar URI autorizada exacta y puerto en Google Cloud.
access_denied / aplicación en pruebas	Agregar la cuenta como usuario de prueba y revisar audiencia.
invalid_client	Comprobar Client ID, Client Secret y variables de entorno.
No se inicia Spring Boot	Verificar Java, dependencias Maven y configuración de ejecución.
Whitelabel Error Page / 404	Revisar nombres de plantillas, rutas y paquetes.
Puerto 8080 ocupado	Detener el proceso o ajustar puerto y redirect URI.
403 al cerrar sesión	Usar POST con Thymeleaf y mantener CSRF habilitado.

14. Actividades de profundización

• Agregar estilos con Bootstrap a index.html y perfil.html.

• Crear una ruta /privado protegida y una plantilla con un mensaje de acceso autorizado.

• Mostrar el issuer y la fecha de autenticación del ID Token (consultar getIdToken().getIssuer() y getIdToken().getAuthenticatedAt()).

• Comparar conceptualmente ID Token y Access Token: finalidad, audiencia y uso.

15. Preguntas de análisis

• ¿Cuál es la diferencia entre autenticación y autorización?

• ¿Por qué OAuth 2.0 no es por sí mismo un protocolo de autenticación?

• ¿Qué añade OpenID Connect al flujo OAuth 2.0?

• ¿Qué función cumple el ID Token?

• ¿Qué sucede cuando alguien solicita /perfil sin sesión?

• ¿Por qué no se debe publicar el Client Secret?

• ¿Qué diferencia hay entre cerrar sesión en la aplicación y en Google?

16. Entregables y evaluación

Entregar: enlace al repositorio GitHub (sin secretos), archivo README con instrucciones de ejecución, capturas de P01–P06 (sin tokens ni credenciales) y documento breve con respuestas a las siete preguntas.

Criterio	Valor
Proyecto y dependencias	15 %

Configuración de Google	20 %
Seguridad y rutas	25 %
Perfil OIDC	15 %
Pruebas y evidencias	15 %
Análisis conceptual	10 %

17. Referencias

Spring Security, OAuth 2.0 Login: https://docs.spring.io/spring-security/reference/servlet/oauth2/login/core.html

Google Identity, OpenID Connect: https://developers.google.com/identity/openid-connect/openid-connect

Spring Initializr: https://start.spring.io/

Nota académica: la interfaz de Google Cloud puede cambiar; verificar las etiquetas actuales de Google Auth Platform.